

Informe Técnico: Implementación y Resiliencia de Demonios con Systemd

Monitoreo Continuo en Segundo Plano con Recuperación Autónoma ante Señales Críticas

Alumno: Piña Ríos Fernando Emmanuel

Carrera: Ingeniería en Sistemas Computacionales

Institución: Instituto Tecnológico de Mazatlán (ITMAZ)

Materia: Sistemas Operativos / Entornos Linux

1. Planteamiento del Problema

En la arquitectura de los sistemas operativos modernos, existen servicios de alta criticidad que no pueden depender de un cronograma periódico estático (como el proporcionado por `cron`), ni de una sesión interactiva vinculada a un emulador de terminal. Servicios de auditoría de seguridad, balanceo de carga o monitoreo de recursos requieren una presencia continua, ininterrumpida y en segundo plano.

Asimismo, dichos procesos deben exhibir alta tolerancia a fallos: ante caídas imprevistas por agotamiento de recursos o intentos de terminación forzada por parte de administradores o atacantes mediante señales letales como `SIGKILL (kill -9)`, el sistema operativo debe detectar la anomalía de inmediato y levantar nuevamente el proceso sin intervención humana en un intervalo inferior a 5 segundos.

2. Objetivos de la Práctica

- Desarrollar un servicio en segundo plano (demonio) en Linux mediante un script en Bash estructurado en un bucle continuo de supervisión.
- Registrar en tiempo real: fecha y hora exacta con segundos, usuario(s) activo(s) y consumo de memoria RAM del sistema operativo.
- Crear y registrar un servicio unitario en el subsistema `systemd` que garantice el auto-arranque y la persistencia del demonio.
- Configurar políticas de resiliencia mediante las directivas `Restart=always` y `RestartSec=3` para revivir el proceso tras la señal `kill -9` en menos de 5 segundos con un nuevo Identificador de Proceso (PID).

3. Fundamentación Técnica

3.1. Diferencia entre Tareas Programadas (Cron) y Demonios (Daemons)

Mientras que `crontab` instancia un nuevo subshell de manera puntual en intervalos fijos (por ejemplo, cada dos minutos) y muere al terminar su tarea, un **demonio** es un proceso persistente que se ejecuta perpetuamente en el espacio de usuario (o de sistema), desconectado de interfaces gráficas o descriptores estándar (STDIN/STDOUT de terminal).

3.2. Arquitectura de Systemd y Políticas de Reinicio

systemd es el sistema de inicialización y gestor de servicios por excelencia en distribuciones Linux modernas como Ubuntu. A través de archivos de configuración denominados *Units* (.service), permite controlar el ciclo de vida de los procesos:

- **Restart=always:** Indica a systemd que, independientemente de la causa de terminación (código de salida limpio 0, error no controlado o interrupción por señales externas como SIGKILL/SIGTERM), el proceso debe relanzarse de inmediato.
- **RestartSec=3:** Establece un tiempo de espera de 3 segundos antes de instanciar un nuevo subprocesso hijo, satisfaciendo el requerimiento de resucitar en menos de 5 segundos.

3.3. Señales POSIX y el comando Kill -9

En Linux, `kill -9` envía la señal SIGKILL (señal numérica 9). A diferencia de SIGTERM (señal 15), la señal SIGKILL no pasa por los manejadores de eventos (handlers) del programa: es interceptada directamente por el kernel, el cual destruye el espacio de direcciones y el hilo de ejecución al instante. Un demonio común muere definitivamente con esta señal; no obstante, cuando está supervisado por systemd, este detecta la muerte del PID hijo mediante `epoll` / llamadas del sistema y dispara la política de reinicio programada.

4. Metodología e Implementación

Paso #1: Creación del Script del Demonio

Se crea el ejecutable en `/usr/local/bin/demonio_vigilante.sh`. El script ejecuta un ciclo continuo que registra cada 5 segundos la estampa de tiempo, usuario y memoria:

```
sudo bash -c 'cat << "EOF" > /usr/local/bin/demonio_vigilante.sh
#!/bin/bash
LOG_FILE="/var/log/demonio_salud.log"

while true; do
    FECHA=$(date "+%Y-%m-%d %H:%M:%S")
    USUARIO=$(who | awk "{print \$1}" | sort -u | tr "\n" " ")
    [ -z "$USUARIO" ] && USUARIO="ninguno_terminal"
    RAM_INFO=$(free -m | awk "/Mem:/ {print \$3\"MB/\"\$2\"MB (\"int(\$3/\\$2*100)\")\"}")

    echo "[FECHA] | Usuario(s): $USUARIO | RAM Usada: $RAM_INFO" >> "$LOG_FILE"
    sleep 5
done
EOF'
```

Paso #2: Otorgar Permisos de Ejecución

```
sudo chmod +x /usr/local/bin/demonio_vigilante.sh
```

Paso #3: Creación de la Unidad Systemd

Se define el servicio del sistema en `/etc/systemd/system/demonio_salud.service`:

```
sudo bash -c 'cat << "EOF" > /etc/systemd/system/demonio_salud.service
[Unit]
Description=Demonio de Monitoreo Continuo del Sistema
After=multi-user.target

[Service]
Type=simple
ExecStart=/usr/local/bin/demonio_vigilante.sh
Restart=always
RestartSec=3
User=root

[Install]
WantedBy=multi-user.target
EOF'
```

Paso #4: Recarga del Gestor y Activación del Servicio

```
sudo systemctl daemon-reload
sudo systemctl enable demonio_salud.service
sudo systemctl start demonio_salud.service
```

Paso #5: Verificación de Estado y Prueba de Muerte Forzada

Comandos para inspeccionar el PID inicial y proceder a ejecutar el comando de aniquilación forzada:

```
# Inspeccionar PID
sudo systemctl status demonio_salud.service

# Intentar matar al demonio (reemplazar <PID> con el número obtenido)
sudo kill -9 <PID>

# Verificar que revivió en menos de 5 segundos con nuevo PID
sleep 4
sudo systemctl status demonio_salud.service
```

5. Conclusiones

La práctica demuestra la superioridad de systemd como supervisor de procesos ante escenarios de fallas catastróficas. La combinación de scripts continuos en Bash con unidades de servicio permite dotar a cualquier aplicación de nivel de servidor con tolerancia a fallos nativa, manteniendo la vigilancia de métricas del sistema sin requerir la presencia activa de un usuario en terminal.